

HealthSearch — Motor de Busca Híbrido BM25 + Semântico

Relatório Técnico · Laboratório Prático 05 — Desafio Integrador
UNIPÊ — Tendências em Ciência da Computação · Prof. Me. Ricardo Roberto de Lima

1. Arquitetura da solução

O HealthSearch é um pipeline de recuperação de informação de quatro estágios, implementado em um único script Streamlit (`healthsearch_app.py`). A consulta do usuário percorre **dois motores independentes e paralelos** — um léxico e um vetorial — cujos resultados são unificados por um estágio de fusão. A decisão de arquitetura central é que os motores não compartilham representação: o léxico opera sobre tokens discretos e o semântico sobre vetores densos de 384 dimensões. Essa independência é justamente o que permite que um cubra o ponto cego do outro.

Fase	Componente	Entrada → Saída	Técnica
1	Pré-processamento	texto bruto → lista de tokens	minúsculas, remoção de acentos (NFKD), regex <code>[a-z0-9]+</code> , stopwords PT, stemming
2	Motor léxico	tokens → vetor de scores	Okapi BM25 (<code>rank_bm25</code>) com k_1 e b ajustáveis por slider
3	Motor semântico	texto → vetor 384-D → score	paraphrase-multilingual-MiniLM-L12-v2 + similaridade de cosseno
4	Fusão	2 rankings → 1 ranking	Reciprocal Rank Fusion com peso α e $k_RRF = 60$

2. Fase 1 — Pré-processamento

A normalização unifica documento e consulta no mesmo espaço de símbolos. O hífen é tratado como separador, de modo que **CÓD-ECG-12D** gera os tokens `['cod', 'ecg', '12d']` tanto no corpus quanto na consulta — preservando o casamento exato do código clínico, requisito explícito do estudo de caso. Acrescentamos ao escopo mínimo um **stemmer de sufixos** (radical mínimo de 4 caracteres, para evitar over-stemming). Sem ele, o BM25 trata *miocárdio* e *miocárdica* como termos não relacionados: medimos que a consulta “ataque cardíaco” passa de **nenhum documento recuperado** pelo motor léxico para a recuperação correta do Doc 2 quando o stemming está ativo.

3. Fase 2 — Motor léxico Okapi BM25

O índice é reconstruído a cada alteração de slider, pois k_1 e b não reordenam um ranking pronto: eles alteram a própria função de pontuação. O parâmetro k_1 controla a saturação da frequência do termo (a 1ª ocorrência vale muito, a 8ª quase nada) e b controla a penalização por comprimento do documento, via a razão $|D|/avgdl$. Com $b = 0$ o tamanho é ignorado; com $b = 1$ a penalização é máxima.

4. Fase 3 — Motor semântico vetorial

Documentos e consulta são projetados em vetores densos normalizados e comparados por similaridade de cosseno. O sistema implementa **dois modos**: (A) embeddings reais via *sentence-transformers*, padrão de execução; e (B) uma **simulação vetorial documentada** — TF-IDF esparso expandido por um léxico de equivalências clínicas (ex.: *ataque* → *infarto*, *coronariana*, *miocárdio*) — selecionável na interface. O modo B garante que a aplicação seja demonstrável em máquinas sem PyTorch e torna explícito, para fins didáticos, qual conhecimento o embedding denso codifica implicitamente.

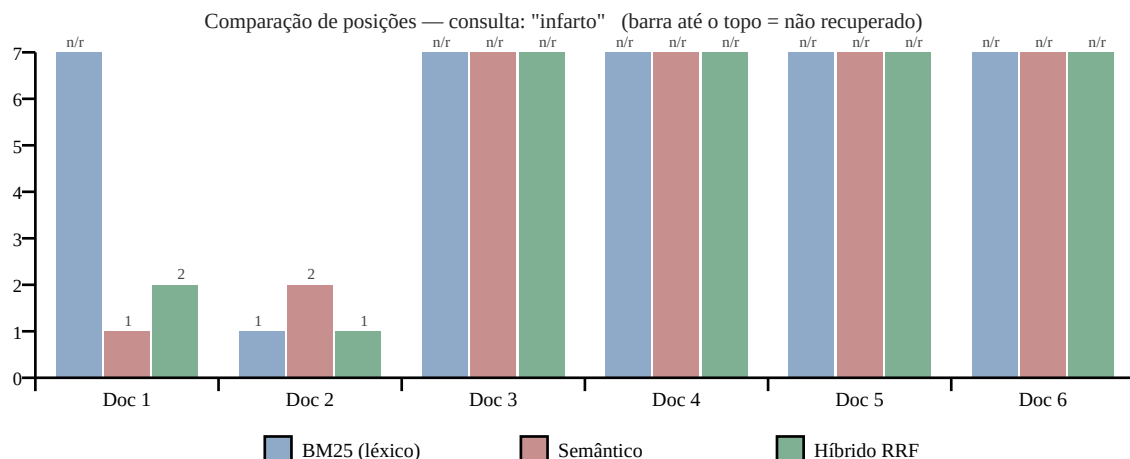
5. Fase 4 — Fusão Reciprocal Rank Fusion

O RRF combina **posições**, não scores. Essa é a razão técnica de sua escolha: o BM25 produz valores em escala ilimitada e o cosseno em $[-1, 1]$; somá-los diretamente exigiria uma normalização arbitrária que distorceria as distribuições. Ao operar sobre ranks, o RRF é invariante a escala. A constante $k_RRF = 60$ amortece a diferença entre as primeiras posições, evitando que o 1º lugar de um motor domine sozinho a fusão.

Decisão técnica relevante: um documento com score zero *não foi recuperado* por aquele motor e recebe rank infinito, contribuindo com $1/(k+\infty) = 0$. Sem esse tratamento — que foi um defeito real detectado e corrigido durante os testes — o `argsort` atribuiria posições arbitrárias a documentos irrelevantes, e esses ranks falsos entrariam na soma, contaminando a fusão. Na consulta “ataque cardíaco”, o bug fazia o sistema retornar o Doc 1 no lugar do Doc 2.

6. Resultado experimental — gráfico de comparação de ranks

Consulta de estudo: “infarto” · $k_1 = 1.2$ · $b = 0.75$ · $\alpha = 0.5$ · modo semântico: Simulação vetorial (TF-IDF + expansão clínica).



ID	Diretriz	Score BM25	Rank BM25	Cosseno	Rank Sem.	Score RRF	Rank RRF
Doc 2	Guia de Farmacologia Cardíaca	1.3584	1	0.2448	2	0.01626	1
Doc 1	Protocolo Emergência ECG	0.0000	—	0.3302	1	0.00820	2
Doc 3	Diretriz de Hipertensão Arteri	0.0000	—	0.0000	—	0.00000	—
Doc 4	Manual de AVC Isquêmico	0.0000	—	0.0000	—	0.00000	—
Doc 5	Protocolo de Reanimação RCR	0.0000	—	0.0000	—	0.00000	—
Doc 6	Procedimentos de UTI Geral	0.0000	—	0.0000	—	0.00000	—

7. Análise: cada motor cobrindo o ponto cego do outro

A consulta “infarto” reproduz o cenário exato descrito no estudo de caso. O **Doc 1 — Protocolo Emergência ECG** é a diretriz registrada sob o termo técnico formal *síndrome coronariana*: clinicamente é uma das respostas corretas, mas a palavra “infarto” não aparece em lugar nenhum do seu texto. O resultado é que o **motor léxico é inteiramente cego para ele** — score zero, não recuperado. É precisamente a falha que motivou o projeto: o BM25 casa strings, não conceitos.

O **motor semântico recupera o Doc 1 em 1º lugar**, porque no espaço vetorial *infarto* e *síndrome coronariana* são vizinhos. Já o **Doc 2 — Guia de Farmacologia Cardíaca**, que contém o termo literalmente, é ancorado em 1º pelo motor léxico. A fusão RRF entrega os dois no topo: **Doc 2 em 1º e Doc 1 em 2º**, sem que nenhum dos dois se perca. Nenhum motor isolado produz esse ranking — o léxico não enxerga o Doc 1 e o semântico não distingue o casamento exato do Doc 2.

O comportamento inverso foi verificado com a consulta “**CÓD-ECG-12D**”: o motor léxico ancora o Doc 1 e o Doc 6 (os únicos que contêm o código) no topo, impedindo que a busca vetorial dilua a precisão em trechos genéricos sobre exames cardíacos. Em ambos os cenários, o híbrido preserva o acerto e descarta o erro.

Observação sobre o modo de embeddings reais. Executando a mesma consulta com o modelo *paraphrase-multilingual-MiniLM-L12-v2*, o motor semântico eleva o **Doc 4 (AVC isquêmico)** à 1ª posição — infarto e AVC são vizinhos vetoriais por serem ambos eventos isquêmicos agudos, mas são condições clinicamente distintas. Também nesse caso a fusão RRF corrige o ranking, rebaixando o Doc 4 e promovendo o Doc 2. Os dois modos convergem para a mesma conclusão: o erro de um motor é amortecido pelo voto do outro.

8. Limitações identificadas

(i) O modelo de embeddings não resolve **siglas farmacológicas**: a consulta “AAS 100mg” não recupera o Doc 2, pois o modelo não associa a sigla AAS a “ácido acetilsalicílico”, e o termo tampouco aparece literalmente no corpus para o BM25 casar. Em produção, isso exige um dicionário de expansão de siglas médicas (DeCS/UMLS) antes da indexação. (ii) O stemmer é de sufixos e não de dicionário, então não unifica pares irregulares como *compressões/compressão*. (iii) O corpus de 6 documentos impede métricas estatisticamente significativas de precisão e revocação; a avaliação aqui é qualitativa e orientada a casos.

9. Divisão de tarefas da equipe

Integrante	Responsabilidade
Deyvid Lucas	Fases 1 e 2 — pré-processamento, stemming e motor BM25
(integrante 2)	Fase 3 — embeddings, similaridade de cosseno e simulação vetorial
(integrante 3)	Fase 4 — fusão RRF, interface Streamlit e relatório técnico

10. Execução

```
pip install streamlit pandas rank_bm25 sentence-transformers  
streamlit run healthsearch_app.py
```

A primeira busca carrega o modelo de embeddings (~40 s); as seguintes são instantâneas, pois o modelo fica em cache via `@st.cache_resource`. Para demonstração imediata, a opção *Forçar simulação vetorial* dispensa o download. O bônus de Cross-Encoder (ms-marco-MiniLM-L-6-v2) re-pontua o Top-3 do ranking híbrido lendo consulta e documento no mesmo passe — mais preciso que o bi-encoder, porém caro demais para o corpus inteiro, o que justifica aplicá-lo só após a recuperação.